

Inspire

Google-Aesthetic Architecture Blueprint & AI Prompt Guide

System Overview

This document serves as the Master AI Prompt Sequence for building the **Inspire** platform. The system is designed to transform 5 core freshman software engineering courses into highly inspirational, career-aligned modules.

Aesthetic Guidelines: The entire application MUST adhere to a "Google Product" aesthetic—utilizing Material Design principles. This means employing a clean white/light-gray canvas (`#f8f9fa`), abundant whitespace, Roboto typography, subtle box shadows (`elevation` system), and the four core Google colors (Blue, Red, Yellow, Green) for accents, progress bars, and call-to-actions.

Future-Proofing Note: The platform architecture is designed anticipating a robust **Admin System** in Phase 2. This upcoming system will handle all content insertion, curriculum updates, and user management. Ensure all data structures (Prisma/Mongoose schemas) are strictly normalized to allow for a headless CMS connection later.

Required Tech Stack & System Purpose

Technology	Primary Use in "Inspire"
Next.js (React)	Frontend framework providing fast SSR, routing, and a scalable UI architecture.
TypeScript	Ensures strict type-safety across both frontend components and backend controllers.
Tailwind CSS	

	Utility styling configured strictly to match Google's Material Design system.
Three.js / Canvas API	Powers the interactive "Inspiration Triggers" (e.g., C Memory map, Physics simulation).
Framer Motion	Provides smooth, native-feeling Material transitions and micro-interactions.
Node.js (ES Modules)	Backend runtime environment exclusively using <code>import`/`export`</code> syntax.
PostgreSQL (Prisma)	Relational DB mapping Core Subjects to specific Career Paths and User Data.
MongoDB (Mongoose)	Flexible NoSQL DB storing unstructured student project notes, failure logs, and code blocks.
Redis	High-speed, in-memory datastore for 5-minute OTP lifecycles and real-time session tracking.
Socket.io	Enables real-time peer-to-peer communication for the "Goal Squads" feature.
Argon2id	State-of-the-art cryptographic hashing algorithm for securing user credentials.

AI Prompt Sequence: Step-by-Step Construction

To the developer: Feed these prompts one by one to your AI coding assistant (e.g., Gemini Flash/Pro) to build the platform module by module without cognitive overload.

STEP 1

Prompt 1: Project Initialization & Material UI Setup

Pass this prompt to the AI to set up the scaffolding, DevOps scripts, and global CSS.

```
ACT AS: Lead Full-Stack Architect.
SYSTEM: "Inspire" - A freshman engineering platform.

OBJECTIVE: Initialize the Next.js frontend, Node.js backend, and local DevOps
scripts using a strict Material Design aesthetic.

TECHNICAL INSTRUCTIONS:
1. Generate the `package.json` for a Node.js backend enforcing `"type":
"module"`.
2. Write a `manager.sh` automation script for local development. CRITICAL:
Include a conditional `wslpath` conversion block so the script seamlessly maps
Windows paths to the Debian environment.
3. Provide the `.env.example` defining endpoints for PostgreSQL, MongoDB, and
Redis. Do not hardcode any credentials in subsequent code.

UI/UX DIRECTIVES (MATERIAL DESIGN):
1. Configure `tailwind.config.js` with Google's color palette:
  - primary: '#4285F4', danger: '#EA4335', warning: '#FBBC05', success:
'#34A853', bgLight: '#f8f9fa'.
2. Define a global CSS file that imports the 'Roboto' font for body text and
'Pacifico' for the main platform logo/title ("Inspire").
3. Create a reusable Material Card CSS class with precise box-shadows: `box-
shadow: 0 1px 2px 0 rgba(60,64,67,0.3), 0 1px 3px 1px rgba(60,64,67,0.15);`.

DELIVERABLES: package.json, manager.sh, .env.example, tailwind.config.js, and
globals.css. Output code only, fully commented.
```

STEP 2

Prompt 2: The Secure Authentication & OTP Engine

Pass this prompt to build the secure entry point with precise micro-interactions.

ACT AS: Identity & Security Engineer.

SYSTEM: "Inspire" - Authentication Module.

OBJECTIVE: Build a secure, Argon2id + Redis OTP authentication flow with a highly polished Google-aesthetic UI.

TECHNICAL INSTRUCTIONS (BACKEND):

1. Create `auth.controller.js` using ES Modules.
2. Implement Argon2id for password hashing during signup.
3. Implement the OTP logic: Generate a 6-digit code, store it in Redis with exactly a 5-minute TTL (`EX 300`), and create a verification endpoint.

UI/UX DIRECTIVES (FRONTEND):

1. Build a sleek `AuthCard.tsx` component (white card, `#f8f9fa` background, rounded corners).
2. LOGIC FLOW: When the user submits valid email/password credentials, the UI MUST immediately transition to a "Success State" showing a Google-green (`#34A853`) checkmark animation (using Framer Motion).
3. Only AFTER the checkmark animation finishes (e.g., 800ms), transition the view to the OTP input screen.

DELIVERABLES: `auth.controller.js`, `redis.client.js`, `AuthCard.tsx` (React). Ensure code is modular and ready to drop into a Next.js app.

STEP 3

Prompt 3: The "Career North Star" Personalization Engine

Pass this to build the relational mapping between boring courses and dream jobs.

ACT AS: Database Modeler & Frontend Architect.

SYSTEM: "Inspire" - Career Personalization Engine (Module A).

OBJECTIVE: Create the core database schema and the onboarding flow where students select their career paths, generating a personalized dashboard.

TECHNICAL INSTRUCTIONS (BACKEND & DB):

1. Write a Prisma schema (`schema.prisma`) for PostgreSQL.
2. Models needed: `User`, `CareerPath` (e.g., AI Engineer, Game Dev), `CoreSubject` (Discrete Math, C, DSA, Physics, Circuit Theory), and a relational mapping model `CareerSubjectRelevance` which stores the specific "Why you need this" text for the relationship.
3. Note: Ensure the schema is normalized so a future Admin Panel can easily perform CRUD operations on `CareerPath` and `CoreSubject`.

UI/UX DIRECTIVES (FRONTEND):

1. Build `OnboardingFlow.tsx`. Use a clean, grid-based layout for career selection.
2. When a career is clicked, use Framer Motion to expand the selection and display the "Mapped Courses" dashboard. The UI should look like Google Workspace (clean, chip-based design, ample padding).

DELIVERABLES: schema.prisma, and the OnboardingFlow.tsx React component.

STEP 4

Prompt 4: The 5 Core "Project-First" Interactive Modules

Pass this prompt to build the actual learning interfaces using interactive canvases.

ACT AS: EdTech UX Engineer & Backend Dev.

SYSTEM: "Inspire" - The Core Curriculum Modules (Module B).

OBJECTIVE: Build the dynamic interfaces for the 5 core subjects, focusing on interactivity over static text.

TECHNICAL INSTRUCTIONS:

1. Define a Mongoose schema (``ProjectData.js``) for MongoDB to store student-generated code snippets and interactive project states.
2. Build ``CourseRenderer.tsx``. This component must dynamically load one of five interactive UI states based on the subject:
 - Discrete Math: A Boolean Logic gate drag-and-drop area.
 - C Programming: A simulated "Memory Map" UI showing stack/heap blocks.
 - DSA: An algorithm speed-comparison chart component.
 - Physics: A basic HTML5 Canvas for drawing kinematics trajectories.
 - Circuit Theory: A clean, SVG-based virtual breadboard interface.
3. Integrate ``KaTeX`` or ``react-katex`` so all formulas in the curriculum text render beautifully in professional LaTeX format.

UI/UX DIRECTIVES:

Embed these interactive widgets inside clean Material cards. Use Google Blue (#4285F4) for primary interactive buttons.

DELIVERABLES: `ProjectData.js` (Mongoose schema), `CourseRenderer.tsx`, and the KaTeX wrapper component.

STEP 5

Prompt 5: The "Resilience" Failure-Recovery Engine

Pass this to build the state-management logic that catches students before they quit.

ACT AS: State Management Expert & UX Designer.

SYSTEM: "Inspire" - Failure-Recovery Engine (Module C).

OBJECTIVE: Build the state logic that monitors student struggles and triggers a supportive intervention.

TECHNICAL INSTRUCTIONS:

1. Implement a Zustand store (`useStudentState.ts`) to track `failedAttempts` on any given module.
2. Set a trigger threshold: If `failedAttempts >= 2`, trigger the `isStruggling` state to true.

UI/UX DIRECTIVES:

1. Build `ResilienceModal.tsx`. When `isStruggling` becomes true, smoothly render a Material dialog centered on screen with a backdrop blur.
2. The modal should NOT use red/error colors. It should use Google Yellow (`#FBB03B`) for an encouraging tone.
3. The content of the modal should present a "Path Correction" (e.g., "Fun fact: The creator of Node.js struggled with this C concept too"). It must contain a button to "Unlock Simplified View."

DELIVERABLES: `useStudentState.ts` (Zustand logic) and `ResilienceModal.tsx`.

STEP 6

Prompt 6: Gamification & Peer Matchmaking (Goal Squads)

Pass this prompt to finalize the system with progression and real-time social features.

ACT AS: Systems Integration & Real-time Engineer.

SYSTEM: "Inspire" - Gamification & Collaboration (Module D).

OBJECTIVE: Implement visual progression and real-time student matching.

TECHNICAL INSTRUCTIONS:

1. Set up a basic `Socket.io` server implementation in Node.js attached to the ES Module backend.
2. Create a "Matchmaking Logic" block: When a user hits the "Find Study Partner" button, query Redis for active sessions matching the exact same `CareerPath` and currently active `CoreSubject` ID.
3. If a match is found, assign them to a Socket.io room (`room:courseID-careerID`).

UI/UX DIRECTIVES:

1. Build a sticky top navigation bar featuring a Material UI Progress Bar (thick, rounded edges, filling up with Google Green #34A853) that increments as the student finishes micro-projects.
2. Build `GoalSquadChat.tsx`, a clean, minimalist chat interface resembling Google Chat (bubbles with subtle shadows, timestamp metadata).

DELIVERABLES: socket.server.js, Matchmaking logic block, TopNavbar.tsx (with progress bar), and GoalSquadChat.tsx.

Ready to Build

Pass Prompt 1 to the AI to begin initialization of the Inspire platform.